

```
# Instalação de pacotes necessários
pkgs <- c(
  "tidymodels",
  'glmnet',
  "rstanarm"
)

instalar <- pkgs[!pkgs %in% installed.packages()[, "Package"]]

if (length(instalar) > 0) {
  install.packages(instalar)
}
```

```
library(tidyverse)
library("tidymodels")
library('janitor')
library('ggcorrplot')
library(workflowsets)
library(rstanarm)
library(glmnet)
```

```
Attaching core tidyverse packages          tidyverse 2.0.0
dplyr      1.2.1      readr      2.2.0
forcats    1.0.1      stringr   1.6.0
ggplot2    4.0.3      tibble    3.3.1
lubridate  1.9.5      tidyr     1.3.2
purrr      1.2.1
```

```
Conflicts                                tidyverse_conflicts()
dplyr::filter() masks stats::filter()
dplyr::lag()    masks stats::lag()
```

Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become

```
Attaching packages          tidymodels 1.5.0
```

```
broom      1.0.12      rsample     1.3.2
dials      1.4.3      tailor     0.1.0
infer      1.1.0      tune       2.1.0
modeldata  1.5.1      workflows  1.3.0
parsnip    1.5.0      workflowsets 1.1.1
recipes    1.3.2      yardstick  1.4.0
```

```
Conflicts                                tidymodels_conflicts()
scales::discard() masks purrr::discard()
```

```
dplyr::filter()   masks stats::filter()
recipes::fixed()  masks stringr::fixed()
dplyr::lag()      masks stats::lag()
yardstick::spec() masks readr::spec()
recipes::step()   masks stats::step()
```

Attaching package: 'janitor'

The following objects are masked from 'package:stats':

```
chisq.test, fisher.test
```

Loading required package: Rcpp

Attaching package: 'Rcpp'

The following object is masked from 'package:rsample':

```
populate
```

This is rstanarm version 2.32.2

- See <https://mc-stan.org/rstanarm/articles/priors> for changes to default priors!
- Default priors may change, so it's safest to specify priors, even if equivalent to the default
- For execution on a local, multicore CPU with excess RAM we recommend calling

```
options(mc.cores = parallel::detectCores())
```

Loading required package: Matrix

Attaching package: 'Matrix'

The following objects are masked from 'package:tidyr':

expand, pack, unpack

Loaded glmnet 5.0

```
#  
set.seed(42)  
  
loan <- read_csv("../data/Loan/loan_approval_dataset.csv") |>  
  clean_names()
```

Rows: 4269 Columns: 13

Column specification

Delimiter: ","

chr (3): education, self_employed, loan_status

dbl (10): loan_id, no_of_dependents, income_annum, loan_amount, loan_term, c...

Use `spec()` to retrieve the full column specification for this data.

Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
#preprocessamento  
loan <- janitor::clean_names(loan)  
  
loan <- loan |>  
  dplyr::filter(residential_assets_value >= 0)
```

```
loan_num <- loan |>  
  dplyr::select(where(is.numeric))  
  
cor_matrix <- cor(  
  loan_num,  
  use = "complete.obs",  
  method = "pearson"  
)  
  
ggcorrplot(  
  cor_matrix,  
  type = "lower",  
  lab = TRUE,  
  lab_size = 4,
```

```

    colors = c("#b2182b", "white", "#2166ac"),
    outline.col = "gray70"
) +
theme_minimal(base_size = 14) +
theme(
  axis.text.x = element_text(
    angle = 90,
    hjust = 1,
    vjust = 0.5
  ),
  axis.text.y = element_text(size = 8)
)

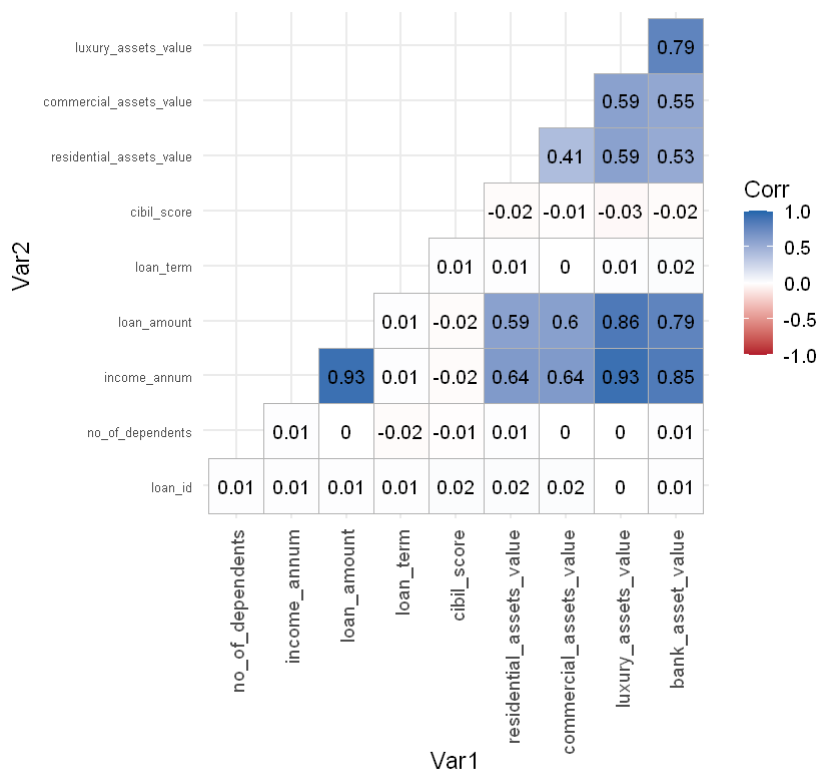
```

Warning message:

```

"`aes_string()` was deprecated in ggplot2 3.0.0.
Please use tidy evaluation idioms with `aes()`.
See also `vignette("ggplot2-in-packages")` for more information.
The deprecated feature was likely used in the ggcorrplot package.
Please report the issue at <https://github.com/kassambara/ggcorrplot/issues>."

```



```
vars_categoricas <- c(
  "education",
  "self_employed"
)

vars_numericas <- c(
  "no_of_dependents",
  "income_annum",
  "loan_amount",
  "loan_term",
  "cibil_score",
  "residential_assets_value",
  "commercial_assets_value",
  "luxury_assets_value",
  "bank_asset_value"
)

# One-Hot Encoding
```

```

loan_encoded <- loan %>%
  mutate(
    education = factor(education),
    self_employed = factor(self_employed)
  ) %>%
  fastDummies::dummy_cols(
    select_columns = vars_categoricas,
    remove_selected_columns = TRUE,
    remove_first_dummy = TRUE
  )

# Padronização (Standardization)
loan_encoded[vars_numericas] <- lapply(
  loan_encoded[vars_numericas],
  function(x) as.numeric(as.character(x))
)
loan_encoded[vars_numericas] <- scale(loan_encoded[vars_numericas])

```

```

#remodendo loan id
loan_encoded <- loan_encoded |>
  dplyr::select(-loan_id)

loan_encoded$loan_status <- ifelse(
  loan_encoded$loan_status %in% c("Approved", 1, "1"),
  1, 0
)
loan_encoded$loan_status <- as.factor(loan_encoded$loan_status)

split <- initial_split(
  loan_encoded,
  prop = 0.75,
  strata = loan_status
)

train_data <- training(split)
test_data <- testing(split)

```

```

count(train_data, loan_status)
count(test_data, loan_status)

```

A tibble: 2 × 2

loan_status <fct>	n <int>
0	1200
1	1980

A tibble: 2 × 2

loan_status <fct>	n <int>
0	401
1	660

para o frequentista - Modelo geral (tudo) OK - Modelo apenas com score Ok - Modelo sem score ok - Modelo com remoção das feats correlacionadas ok - Modelo com PCA - Modelo com regularização para selecionar feats (glmnet) ok

```
rec_lasso <- recipe(loan_status ~ ., data = train_data) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_normalize(all_numeric_predictors())

logit_lasso_spec <- logistic_reg(
  penalty = tune(),
  mixture = 1      # LASSO puro
) %>%
  set_engine("glmnet")

wf_lasso <- workflow() %>%
  add_recipe(rec_lasso) %>%
  add_model(logit_lasso_spec)

folds <- vfold_cv(train_data, v = 5)

grid_lambda <- grid_regular(penalty(), levels = 50)

lasso_res <- tune_grid(
  wf_lasso,
  resamples = folds,
  grid = grid_lambda,
  metrics = metric_set(roc_auc)
)
best_lambda <- select_best(lasso_res, metric = "roc_auc")
best_lambda
```

A tibble: 1 × 2

penalty <dbl>	.config <chr>
0.0008685114	pre0_mod35_post0

```
final_lasso_wf <- finalize_workflow(  
  wf_lasso,  
  best_lambda  
)  
  
final_lasso_fit <- fit(  
  final_lasso_wf,  
  data = train_data  
)  
  
lasso_coefs <- final_lasso_fit %>%  
  extract_fit_parsnip() %>%  
  tidy()  
  
vars_selected <- lasso_coefs %>%  
  filter(term != "(Intercept)") %>%  
  filter(estimate != 0) %>%  
  mutate(abs_est = abs(estimate))  
vars_importance <- vars_selected %>%  
  mutate(importance = abs_est / sum(abs_est)) %>%  
  select(variable = term, importance) %>%  
  arrange(desc(importance))  
  
vars_importance
```

A tibble: 10 × 2

variable <chr>	importance <dbl>
cibil_score	0.540896692
income_annum	0.164836108
loan_amount	0.137832359
loan_term	0.108517830
bank_asset_value	0.014808588
luxury_assets_value	0.010021713
commercial_assets_value	0.008276973
no_of_dependents	0.006634555

variable <chr>	importance <dbl>
self_employed_Yes	0.004926413
education_Not Graduate	0.003248769

```
not_best_vars <- vars_importance %>%
  dplyr::filter(importance < 0.02) %>%
  dplyr::pull(variable)
```

```
#modelo geral
rec_all <- recipe(loan_status ~ ., data = train_data)

# score only
rec_score_only <- recipe(
  loan_status ~ cibil_score,
  data = train_data
)

#resultado da regularização
rec_feat_select <- recipe(
  loan_status ~ .,
  data = train_data
) %>%
  step_rm(all_of(not_best_vars))

#no score
rec_no_score <- recipe(
  loan_status ~ .,
  data = train_data
) |>
  step_rm(cibil_score)

# Removendo alta correlação
rec_no_assets <- recipe(
  loan_status ~ .,
  data = train_data
) |>
  step_rm(luxury_assets_value, bank_asset_value)

#com pca
```

```

rec_pca <- recipe(
  loan_status ~ .,
  data = train_data
) |>
  step_normalize(all_numeric_predictors()) |>
  step_pca(
    all_numeric_predictors(),
    threshold = 0.95
  )

```

```

log_spec <- logistic_reg() |>
  set_engine("glm") |>
  set_mode("classification")

```

```

wf_logit <- workflow_set(
  preproc = list(
    all_features = rec_all,
    score_only   = rec_score_only,
    no_score     = rec_no_score,
    no_assets    = rec_no_assets,
    feat_select  = rec_feat_select
  ),
  models = list(
    logit = log_spec
  )
)

```

```

wf_pca <- workflow_set(
  preproc = list(
    pca = rec_pca
  ),
  models = list(
    logit = log_spec
  )
)

```

```

wf_set <- rbind(
  wf_logit,
  wf_pca
)

```

```

wf_fitted <- wf_set %>%
  mutate(
    fit = purrr::map(
      info,
      ~ fit(.x$workflow[[1]], data = train_data)
    )
  )

```

```

wf_preds <- wf_fitted %>%
  mutate(
    preds = purrr::map(
      fit,
      ~ predict(.x, test_data, type = "prob") %>%
        bind_cols(
          predict(.x, test_data, type = "class"),
          test_data %>% select(loan_status)
        )
    )
  )

```

```

fits <- wf_set %>%
  mutate(
    fit = purrr::map(
      info,
      ~ fit(.x$workflow[[1]], data = train_data)
    )
  )

```

```

library(dplyr)
library(purrr)
library(yardstick)

preds <- fits %>%
  mutate(
    preds = purrr::map(
      fit,
      ~{
        prob_tbl <- predict(.x, test_data, type = "prob")

        tibble(
          truth = test_data$loan_status,
          class = predict(.x, test_data, type = "class")$.pred_class,

```

```

        prob = prob_tbl$.pred_1
      }
    )
  ) %>%
  select(wflow_id, preds)

```

```

preds <- preds %>%
  tidyr::unnest(preds)

```

```

metrics_table <- preds %>%
  group_by(wflow_id) %>%
  summarise(

    # classificação geral
    accuracy = accuracy_vec(truth, class),
    bal_accuracy = bal_accuracy_vec(truth, class),

    # ranking / separação
    roc_auc = roc_auc_vec(truth, prob, event_level = "second"),
    log_loss = mn_log_loss_vec(truth, prob),

    # sensibilidade / especificidade
    sens = sens_vec(truth, class),
    spec = spec_vec(truth, class),

    # custo de erro (crédito)
    precision = precision_vec(truth, class),
    npv = npv_vec(truth, class),

    # equilíbrio
    f1 = f_meas_vec(truth, class),
    f2 = f_meas_vec(truth, class, beta = 2),

    # robustez estatística
    kap = kap_vec(truth, class),
    mcc = mcc_vec(truth, class),

    # equilíbrio sens/spec
    gmean = sqrt(sens_vec(truth, class) * spec_vec(truth, class)),

    .groups = "drop"
  )

```

)

metrics_table

A tibble: 6 × 14

	ac- cu-	bal_ac- cu-	roc_auc	log_loss	sens	spec	pre- ci-	npv	f1	f2	kap	mcc	gmean
	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
all_features_logit	0.929312	0.240988	0.976270	0.469719	0.274339	0.454501	0.909547	0.411765	0.613297	0.409580	0.944016	0.194600	0.923852
feat_select_logit	0.928369	0.233419	0.976297	0.239070	0.274339	0.439399	0.907268	0.410870	0.500090	0.364153	0.751082	0.187518	0.923115
no_as_sets_logit	0.929312	0.240988	0.976184	0.291829	0.274339	0.454501	0.909547	0.411765	0.613297	0.409580	0.944016	0.194600	0.923852
no_score_logit	0.614510	0.504210	0.597442	0.170660	0.523699	0.956000	0.200006	0.241301	0.931269	0.348207	0.177069	0.288323	0.37588
pca_logit	0.922710	0.163409	0.973600	0.888342	0.902701	0.124201	0.903797	0.933933	0.969801	0.929468	0.351508	0.352101	0.995978
score_only_logit	0.927487	0.235629	0.966913	0.415061	0.707730	0.939393	0.900990	0.436839	0.434798	0.637018	0.588282	0.158989	0.923426

```
extract_model_summary <- function(wf_fit) {  
  model_fit <- workflows::extract_fit_parsnip(wf_fit)  
  
  if (inherits(model_fit$fit, "glm")) {  
    return(summary(model_fit$fit))  
  }  
  
  if (inherits(model_fit$fit, "lognet")) {  
    return(model_fit$fit)  
  }  
  
  NULL  
}
```

```
model_summaries <- wf_fitted %>%  
  mutate(  
    summary = purrr::map(fit, extract_model_summary)  
  ) %>%  
  select(wflow_id, summary)
```

```

for (i in seq_len(nrow(model_summaries))) {
  cat("\n===== \n")
  cat("Modelo:", model_summaries$wflow_id[i], "\n")
  cat("===== \n\n")
  print(model_summaries$summary[[i]])
}

```

```

=====
Modelo: all_features_logit
=====

```

Call:

```
stats::glm(formula = ..y ~ ., family = stats::binomial, data = data)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	1.77048	0.13386	13.227	< 2e-16 ***
no_of_dependents	-0.06320	0.06749	-0.936	0.349
income_annum	-1.73194	0.28940	-5.985	2.17e-09 ***
loan_amount	1.24657	0.18624	6.693	2.18e-11 ***
loan_term	-0.86074	0.07432	-11.582	< 2e-16 ***
cibil_score	4.22438	0.16427	25.716	< 2e-16 ***
residential_assets_value	0.01737	0.08789	0.198	0.843
commercial_assets_value	0.09282	0.08653	1.073	0.283
luxury_assets_value	0.26295	0.18407	1.429	0.153
bank_asset_value	0.19775	0.12455	1.588	0.112
`education_Not Graduate`	-0.07937	0.13514	-0.587	0.557
self_employed_Yes	0.09816	0.13524	0.726	0.468

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 4215.1 on 3179 degrees of freedom
 Residual deviance: 1434.6 on 3168 degrees of freedom
 AIC: 1458.6

Number of Fisher Scoring iterations: 7

```
=====
Modelo: score_only_logit
=====
```

```
Call:
stats::glm(formula = ..y ~ ., family = stats::binomial, data = data)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	1.54175	0.08324	18.52	<2e-16 ***
cibil_score	3.64588	0.13393	27.22	<2e-16 ***

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 4215.1 on 3179 degrees of freedom
Residual deviance: 1642.5 on 3178 degrees of freedom
AIC: 1646.5

Number of Fisher Scoring iterations: 6

```
=====
Modelo: no_score_logit
=====
```

```
Call:
stats::glm(formula = ..y ~ ., family = stats::binomial, data = data)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	0.50646	0.06345	7.982	1.44e-15 ***
no_of_dependents	-0.04991	0.03696	-1.350	0.177
income_annum	-0.44160	0.15344	-2.878	0.004 **
loan_amount	0.43262	0.09832	4.400	1.08e-05 ***
loan_term	-0.21687	0.03715	-5.838	5.30e-09 ***
residential_assets_value	-0.03404	0.04745	-0.717	0.473
commercial_assets_value	0.04281	0.04800	0.892	0.372
luxury_assets_value	-0.04472	0.10085	-0.443	0.657
bank_asset_value	0.03714	0.07051	0.527	0.598

```

`education_Not Graduate` -0.01130    0.07395  -0.153    0.879
self_employed_Yes        0.01445    0.07395   0.195    0.845
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

(Dispersion parameter for binomial family taken to be 1)

```

Null deviance: 4215.1  on 3179  degrees of freedom
Residual deviance: 4155.2  on 3169  degrees of freedom
AIC: 4177.2

```

Number of Fisher Scoring iterations: 4

```

=====
Modelo: no_assets_logit
=====

```

```

Call:
stats::glm(formula = ..y ~ ., family = stats::binomial, data = data)

```

```

Coefficients:
              Estimate Std. Error z value Pr(>|z|)
(Intercept)      1.759474   0.133234  13.206 < 2e-16 ***
no_of_dependents -0.065839   0.067279  -0.979   0.328
income_annum     -1.307910   0.198633  -6.585 4.56e-11 ***
loan_amount       1.240585   0.185779   6.678 2.43e-11 ***
loan_term        -0.853741   0.073937 -11.547 < 2e-16 ***
cibil_score       4.209888   0.163706  25.716 < 2e-16 ***
residential_assets_value 0.008164   0.087495   0.093   0.926
commercial_assets_value 0.095824   0.086228   1.111   0.266
`education_Not Graduate` -0.082798   0.134863  -0.614   0.539
self_employed_Yes  0.109560   0.134880   0.812   0.417
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

```

(Dispersion parameter for binomial family taken to be 1)

```

Null deviance: 4215.1  on 3179  degrees of freedom
Residual deviance: 1439.1  on 3170  degrees of freedom
AIC: 1459.1

```


Number of Fisher Scoring iterations: 7

```
=====
Modelo: feat_select_logit
=====
```

Call:

```
stats::glm(formula = ..y ~ ., family = stats::binomial, data = data)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	1.771024	0.094996	18.643	< 2e-16 ***
income_annum	-1.249009	0.191707	-6.515	7.26e-11 ***
loan_amount	1.244885	0.185253	6.720	1.82e-11 ***
loan_term	-0.852184	0.073687	-11.565	< 2e-16 ***
cibil_score	4.205449	0.163397	25.738	< 2e-16 ***
residential_assets_value	0.004263	0.087197	0.049	0.961

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 4215.1 on 3179 degrees of freedom
Residual deviance: 1442.1 on 3174 degrees of freedom
AIC: 1454.1

Number of Fisher Scoring iterations: 7

```
=====
Modelo: pca_logit
=====
```

Call:

```
stats::glm(formula = ..y ~ ., family = stats::binomial, data = data)
```

Coefficients:

	Estimate	Std. Error	z value	Pr(> z)
(Intercept)	1.73307	0.09303	18.629	<2e-16 ***
PC1	0.04730	0.03141	1.506	0.132

```

PC2          -0.97661      0.07096 -13.763    <2e-16 ***
PC3          -0.60361      0.06778  -8.905    <2e-16 ***
PC4          -3.50307      0.13935 -25.139    <2e-16 ***
PC5          -0.99950      0.07503 -13.321    <2e-16 ***
PC6           1.63189      0.08987  18.158    <2e-16 ***
PC7           0.07480      0.08505   0.880      0.379
PC8          -0.04570      0.09348  -0.489      0.625

```

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

(Dispersion parameter for binomial family taken to be 1)

Null deviance: 4215.1 on 3179 degrees of freedom
Residual deviance: 1486.8 on 3171 degrees of freedom
AIC: 1504.8

Number of Fisher Scoring iterations: 7

metrics_table

A tibble: 6 × 14

	ac- cu-	bal_ac- cu-	roc_auc	log_loss	sens	spec	sion	npv	f1	f2	kap	mcc	gmean
	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
all_features_logit	0.929312	0.240988	0.762720	0.469719	0.274334	0.454501	0.095477	0.411769	0.061320	0.409580	0.944061	0.194600	0.238520
feat_sets_logit	0.928369	0.233419	0.762970	0.239070	0.274334	0.439399	0.072682	0.410870	0.050000	0.036418	0.375108	0.275187	0.231115
lect_logit	0.929312	0.240988	0.761845	0.291829	0.274334	0.454501	0.095477	0.411769	0.061320	0.409580	0.944061	0.194600	0.238520
sets_logit	0.614571	0.504210	0.597442	0.170660	0.523699	0.560062	0.200000	0.241301	0.593126	0.634820	0.701770	0.692883	0.237588
pca_logit	0.922710	0.163409	0.736008	0.883426	0.902701	0.912420	0.903797	0.933933	0.969848	0.929468	0.351508	0.352101	0.995978
score_only_logit	0.927428	0.235629	0.669104	0.150619	0.773067	0.393939	0.009900	0.436839	0.043479	0.063701	0.588282	0.589892	0.234266

```

fits_vif <- fits %>%
  dplyr::filter(
    wflow_id %in% c(
      "all_features_logit",
      "no_assets_logit",

```

```

    "no_score_logit",
    "feat_select_logit"
  )
)

#apenas os modelos sem PCA...

```

```

library(car)

calc_vif <- function(wf_fit) {
  wf_fit %>%
    workflows::extract_fit_parsnip() %>%
    pluck("fit") %>%      # objeto glm
    car::vif()
}

vif_results <- fits_vif %>%
  mutate(
    vif = purrr::map(fit, calc_vif)
  )

```

Loading required package: carData

Registered S3 method overwritten by 'car':

method	from
na.action.merMod	lme4

Attaching package: 'car'

The following object is masked from 'package:rstanarm':

logit

The following object is masked from 'package:dplyr':

recode

The following object is masked from 'package:purrr':

some

```
vif_tidy <- vif_results %>%  
  select(wflow_id, vif) %>%  
  tidyr::unnest_longer(vif, values_to = "vif") %>%  
  rename(variable = vif_id)
```

```
vif_summary <- vif_tidy %>%  
  group_by(wflow_id) %>%  
  summarise(  
    max_vif = max(vif),  
    mean_vif = mean(vif)  
  )  
vif_summary
```

A tibble: 4 × 3

wflow_id <chr>	max_vif <dbl>	mean_vif <dbl>
all_features_logit	18.147188	4.146099
feat_select_logit	8.031679	3.979695
no_assets_logit	8.585693	2.789998
no_score_logit	17.041102	4.216524

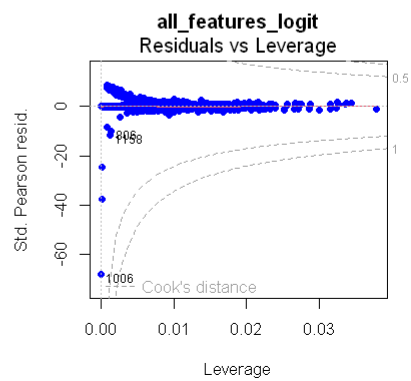
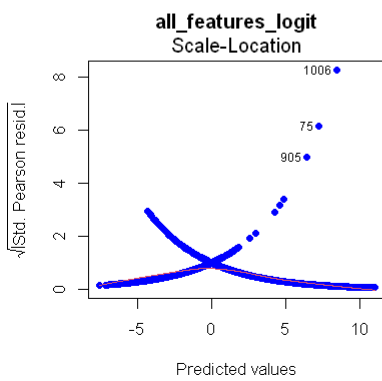
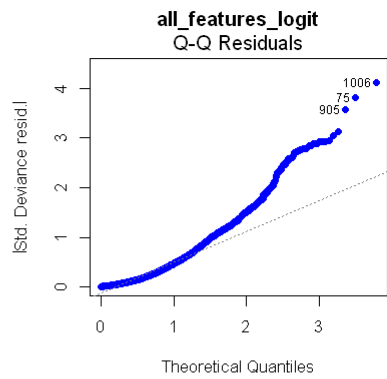
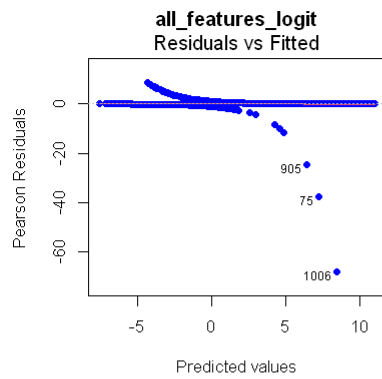
```
get_model <- function(wf_fit) {  
  workflows::extract_fit_parsnip(wf_fit)$fit  
}
```

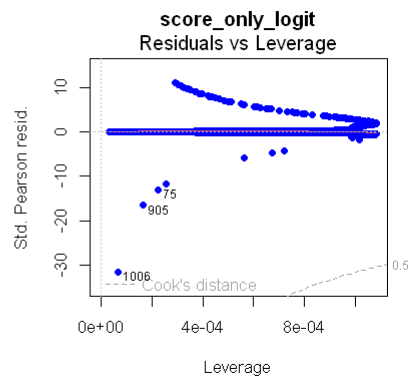
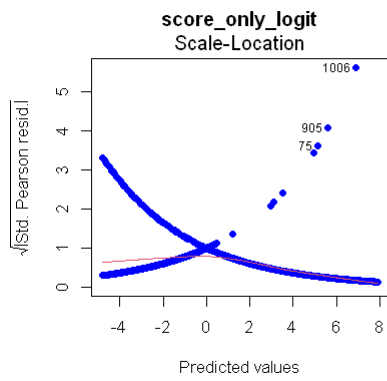
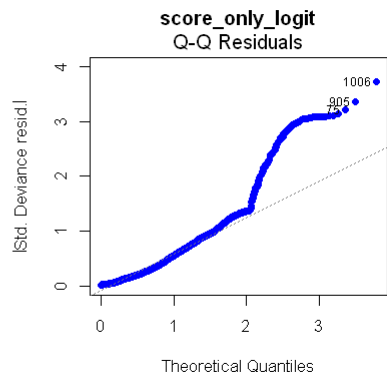
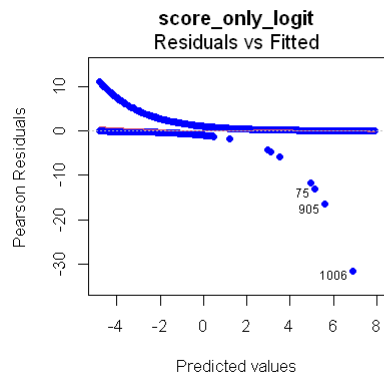
```
purrr::walk2(  
  wf_fitted$fit,  
  wf_fitted$wflow_id,  
  ~ {  
    modelo <- get_model(.x)  
  
    par(mfrow = c(2, 2))  
    plot(  
      modelo,  
      pch = 19,  
      col = "blue",
```

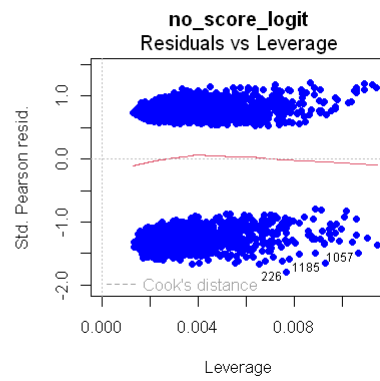
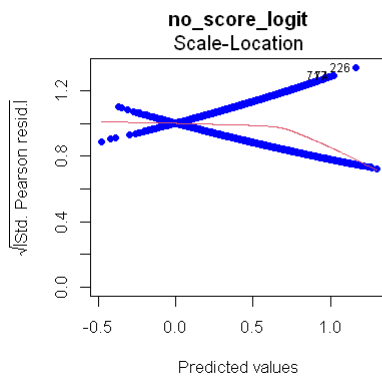
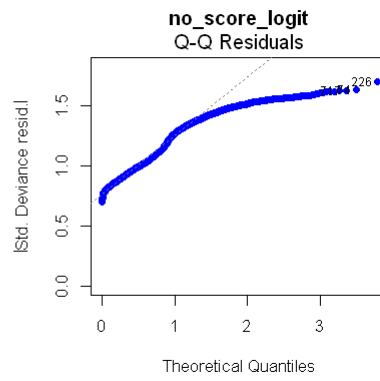
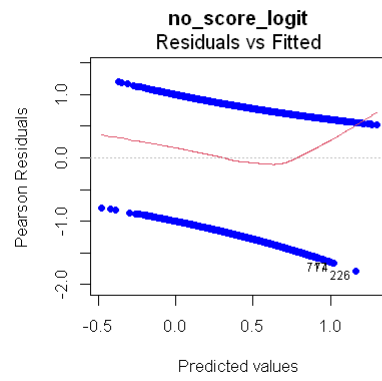
```

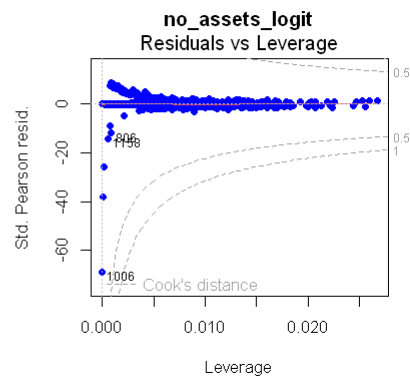
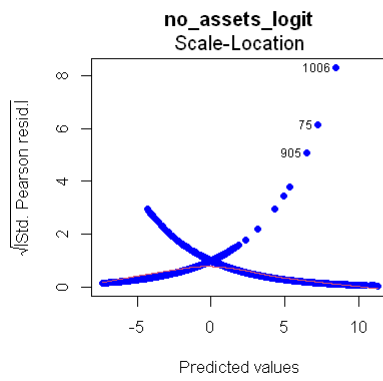
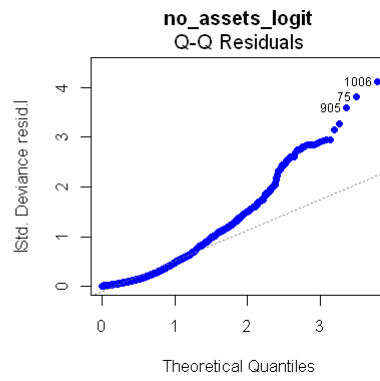
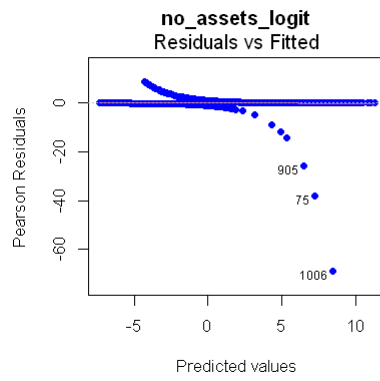
    main = .y
  )
}
)

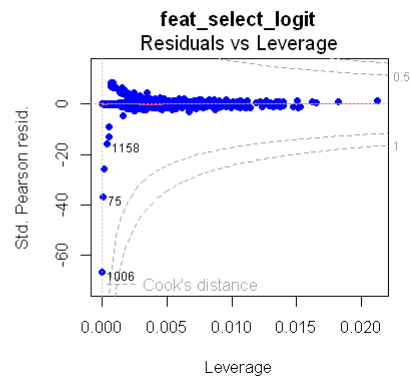
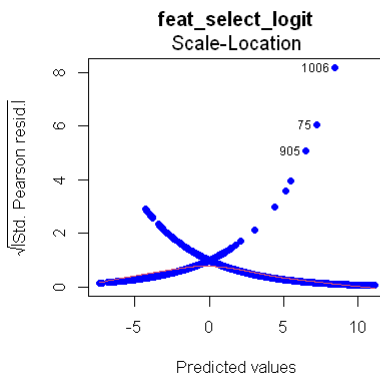
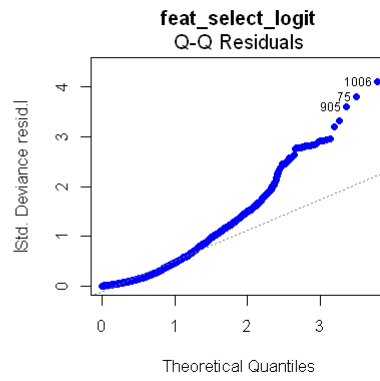
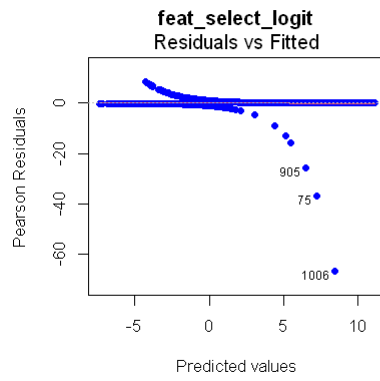
```

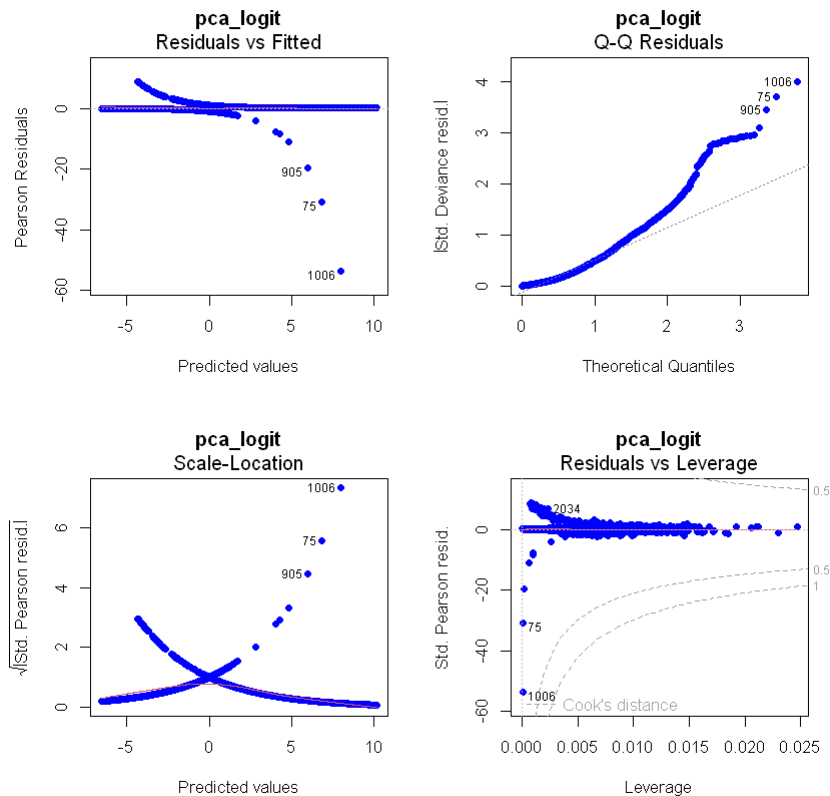












`all_features_logit` não é adequado para inferência, apesar do ótimo desempenho. `no_assets_logit` reduz drasticamente a colinearidade sem perda preditiva. `feat_select_logit` também melhora o VIF, mas menos que `no_assets_logit`.

```
library(dplyr)

model_summary <- metrics_table %>%
  select(
    wflow_id,
    accuracy,
    bal_accuracy,
    roc_auc,
    f1
  ) %>%
  left_join(
    vif_summary %>%
      group_by(wflow_id) %>%
      summarise(
        max_vif = max(max_vif, na.rm = TRUE),
```

```

      mean_vif = mean(mean_vif, na.rm = TRUE)
    ),
    by = "wflow_id"
  )

```

```

library(dplyr)

model_summary <- metrics_table %>%
  left_join(
    vif_summary %>%
      group_by(wflow_id) %>%
      summarise(
        max_vif = max(max_vif, na.rm = TRUE),
        mean_vif = mean(mean_vif, na.rm = TRUE)
      ),
    by = "wflow_id"
  )

```

```
model_summary
```

A tibble: 6 × 16

wflow_id	ac-cu	bal_ac-cu	roc_auc	log_loss	spec	sion	npv	f1	f2	kap	mcc	gmean	max_vif	mean_vif
<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
all_features_logit	0.929302	0.2240988	0.6279016	0.69793	0.274394	0.4546095	0.477117	0.556132	0.6740959	0.4461946	0.923852	0.474884	18.15	6.099
feat_select_logit	0.928369	0.233413	0.629723	0.69070	0.274394	0.4546095	0.477117	0.556132	0.6740959	0.4461946	0.923852	0.474884	18.15	6.099
no_assets_logit	0.929302	0.2240988	0.6279016	0.69793	0.274394	0.4546095	0.477117	0.556132	0.6740959	0.4461946	0.923852	0.474884	18.15	6.099
no_score	0.6145046	0.420458	0.74121	0.70665	0.236935	0.6062000	0.02413	0.49312	0.66348	0.24717	0.766928	0.237583	8.03	1.6524
pca_logit	0.922704	0.163497	0.363888	0.34290	0.274394	0.4546095	0.477117	0.556132	0.6740959	0.4461946	0.923852	0.474884	18.15	6.099
score	0.6145046	0.420458	0.74121	0.70665	0.236935	0.6062000	0.02413	0.49312	0.66348	0.24717	0.766928	0.237583	8.03	1.6524

bom, no_score aponta que o cibil_score é importante — Modelos com melhores desempenhos geral all_features_logit feat_select_logit no_assets_logit - ROC AUC ~ 0.976 - F1 ~ 0.905–0.906 all_features_logit -> Max VIF = 18.15 → forte colinearidade feat_select_logit -> Max VIF = 8.03 → redução significativa no_assets_logit -> melhor equilíbrio entre VIF (média menor) e performance

O PCA não aplica VIF e tem uma queda na performance

```
library(tidymodels)
library(rstanarm)
```

```
library(rstanarm)

fit_bayes <- stan_glm(
  loan_status ~ . - luxury_assets_value - bank_asset_value,
  data = train_data,
  family = binomial(link = "logit"),
  prior = normal(0, 1),
  prior_intercept = normal(0, 5),
  chains = 4,
  iter = 2000,
  seed = 123
)
```

SAMPLING FOR MODEL 'bernoulli' NOW (CHAIN 1).

Chain 1:

Chain 1: Gradient evaluation took 0.000236 seconds

Chain 1: 1000 transitions using 10 leapfrog steps per transition would take 2.36 seconds.

Chain 1: Adjust your expectations accordingly!

Chain 1:

Chain 1:

Chain 1: Iteration: 1 / 2000 [0%] (Warmup)

Chain 1: Iteration: 200 / 2000 [10%] (Warmup)

Chain 1: Iteration: 400 / 2000 [20%] (Warmup)

Chain 1: Iteration: 600 / 2000 [30%] (Warmup)

Chain 1: Iteration: 800 / 2000 [40%] (Warmup)

Chain 1: Iteration: 1000 / 2000 [50%] (Warmup)

Chain 1: Iteration: 1001 / 2000 [50%] (Sampling)

Chain 1: Iteration: 1200 / 2000 [60%] (Sampling)

Chain 1: Iteration: 1400 / 2000 [70%] (Sampling)

Chain 1: Iteration: 1600 / 2000 [80%] (Sampling)

Chain 1: Iteration: 1800 / 2000 [90%] (Sampling)

Chain 1: Iteration: 2000 / 2000 [100%] (Sampling)

Chain 1:

Chain 1: Elapsed Time: 3.023 seconds (Warm-up)

Chain 1: 2.609 seconds (Sampling)

Chain 1: 5.632 seconds (Total)

Chain 1:

SAMPLING FOR MODEL 'bernoulli' NOW (CHAIN 2).

Chain 2:

Chain 2: Gradient evaluation took 0.000148 seconds

Chain 2: 1000 transitions using 10 leapfrog steps per transition would take 1.48 seconds.

Chain 2: Adjust your expectations accordingly!

Chain 2:

Chain 2:

Chain 2: Iteration: 1 / 2000 [0%] (Warmup)

Chain 2: Iteration: 200 / 2000 [10%] (Warmup)

Chain 2: Iteration: 400 / 2000 [20%] (Warmup)

Chain 2: Iteration: 600 / 2000 [30%] (Warmup)

Chain 2: Iteration: 800 / 2000 [40%] (Warmup)

Chain 2: Iteration: 1000 / 2000 [50%] (Warmup)

Chain 2: Iteration: 1001 / 2000 [50%] (Sampling)

Chain 2: Iteration: 1200 / 2000 [60%] (Sampling)

Chain 2: Iteration: 1400 / 2000 [70%] (Sampling)

Chain 2: Iteration: 1600 / 2000 [80%] (Sampling)

Chain 2: Iteration: 1800 / 2000 [90%] (Sampling)

Chain 2: Iteration: 2000 / 2000 [100%] (Sampling)

Chain 2:

Chain 2: Elapsed Time: 3.238 seconds (Warm-up)

Chain 2: 3.084 seconds (Sampling)

Chain 2: 6.322 seconds (Total)

Chain 2:

SAMPLING FOR MODEL 'bernoulli' NOW (CHAIN 3).

Chain 3:

Chain 3: Gradient evaluation took 0.000219 seconds

Chain 3: 1000 transitions using 10 leapfrog steps per transition would take 2.19 seconds.

Chain 3: Adjust your expectations accordingly!

Chain 3:

Chain 3:

Chain 3: Iteration: 1 / 2000 [0%] (Warmup)

Chain 3: Iteration: 200 / 2000 [10%] (Warmup)

Chain 3: Iteration: 400 / 2000 [20%] (Warmup)

Chain 3: Iteration: 600 / 2000 [30%] (Warmup)

Chain 3: Iteration: 800 / 2000 [40%] (Warmup)

Chain 3: Iteration: 1000 / 2000 [50%] (Warmup)

Chain 3: Iteration: 1001 / 2000 [50%] (Sampling)

Chain 3: Iteration: 1200 / 2000 [60%] (Sampling)

Chain 3: Iteration: 1400 / 2000 [70%] (Sampling)

```
Chain 3: Iteration: 1600 / 2000 [ 80%] (Sampling)
Chain 3: Iteration: 1800 / 2000 [ 90%] (Sampling)
Chain 3: Iteration: 2000 / 2000 [100%] (Sampling)
Chain 3:
Chain 3: Elapsed Time: 3.539 seconds (Warm-up)
Chain 3:           2.65 seconds (Sampling)
Chain 3:           6.189 seconds (Total)
Chain 3:
```

SAMPLING FOR MODEL 'bernoulli' NOW (CHAIN 4).

```
Chain 4:
Chain 4: Gradient evaluation took 0.000155 seconds
Chain 4: 1000 transitions using 10 leapfrog steps per transition would take 1.55 seconds.
Chain 4: Adjust your expectations accordingly!
Chain 4:
Chain 4:
Chain 4: Iteration:    1 / 2000 [  0%] (Warmup)
Chain 4: Iteration:   200 / 2000 [ 10%] (Warmup)
Chain 4: Iteration:   400 / 2000 [ 20%] (Warmup)
Chain 4: Iteration:   600 / 2000 [ 30%] (Warmup)
Chain 4: Iteration:   800 / 2000 [ 40%] (Warmup)
Chain 4: Iteration:  1000 / 2000 [ 50%] (Warmup)
Chain 4: Iteration:  1001 / 2000 [ 50%] (Sampling)
Chain 4: Iteration:  1200 / 2000 [ 60%] (Sampling)
Chain 4: Iteration:  1400 / 2000 [ 70%] (Sampling)
Chain 4: Iteration:  1600 / 2000 [ 80%] (Sampling)
Chain 4: Iteration:  1800 / 2000 [ 90%] (Sampling)
Chain 4: Iteration:  2000 / 2000 [100%] (Sampling)
Chain 4:
Chain 4: Elapsed Time: 3.265 seconds (Warm-up)
Chain 4:           3.29 seconds (Sampling)
Chain 4:           6.555 seconds (Total)
Chain 4:
```

```
post_prob <- posterior_linpred(
  fit_bayes,
  newdata = test_data,
  transform = TRUE
)

prob_mean <- colMeans(post_prob)
```

Instead of `posterior_linpred(..., transform=TRUE)` please call `posterior_epred()`, which provides

```
pred_class <- ifelse(prob_mean > 0.5, "yes", "no") %>%  
  factor(levels = levels(test_data$loan_status))
```

```
library(dplyr)  
  
preds_bayes <- test_data %>%  
  mutate(  
    prob = prob_mean,  
    class = factor(  
      ifelse(prob >= 0.5, "1", "0"),  
      levels = levels(loan_status)  
    ),  
    truth = loan_status,  
    wflow_id = "bayes_logit"  
  )  
  
preds_bayes <- preds_bayes %>%  
  mutate(wflow_id = "bayes_logit")
```

```
library(yardstick)  
library(dplyr)  
  
metrics_bayes <- preds_bayes %>%  
  group_by(wflow_id) %>%  
  summarise(  
  
    # classificação geral  
    accuracy = accuracy_vec(truth, class),  
    bal_accuracy = bal_accuracy_vec(truth, class),  
  
    # ranking / separação  
    roc_auc = roc_auc_vec(truth, prob, event_level = "second"),  
    log_loss = mn_log_loss_vec(truth, prob),  
  
    # sensibilidade / especificidade  
    sens = sens_vec(truth, class),  
    spec = spec_vec(truth, class),  
  
    # custo de erro (crédito)  
    precision = precision_vec(truth, class),
```

```

npv = npv_vec(truth, class),

# equilibrio
f1 = f_meas_vec(truth, class),
f2 = f_meas_vec(truth, class, beta = 2),

# robustez estadística
kap = kap_vec(truth, class),
mcc = mcc_vec(truth, class),

# equilibrio sens/spec
gmean = sqrt(
  sens_vec(truth, class) * spec_vec(truth, class)
),

.groups = "drop"
)

```

```
table(preds_bayes$class)
```

```

0    1
396 665

```

```

library(car)

vif_bayes <- car::vif(fit_bayes)
vif_bayes

```

```

no_of_dependents 1.01283844292457income__annum
8.52743199251324loan__amount
7.47191969736223loan__term
1.21569621978444cibil__score
1.24925810581672residential__assets__value
1.71551136810464commercial__assets__value
1.6229628986236education_Not Graduate
1.00818025806458self_employed__Yes
1.00391026977515

```



```
vif_summary <- tibble(
  max_vif = max(vif_bayes, na.rm = TRUE),
  mean_vif = mean(vif_bayes, na.rm = TRUE)
)

metrics_bayes <- bind_cols(metrics_bayes, vif_summary)
```

```
metrics_all <- bind_rows(
  model_summary,
  metrics_bayes
)

metrics_all
```

A tibble: 7 × 16

	ac- cu-	bal_ac- cu-	roc_auc	log_loss	spec	sion	npv	f1	f2	kap	mcc	gmean	max_vif	mean_vif
<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
all_features_logit	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022
feat_select_logit	0.9283692	0.9283692	0.9283692	0.9283692	0.9283692	0.9283692	0.9283692	0.9283692	0.9283692	0.9283692	0.9283692	0.9283692	0.9283692	0.9283692
no_as_sets_logit	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022
no_score_logit	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022
pca_logit	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022
score_logit	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022
bayes_logit	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022	0.9293022

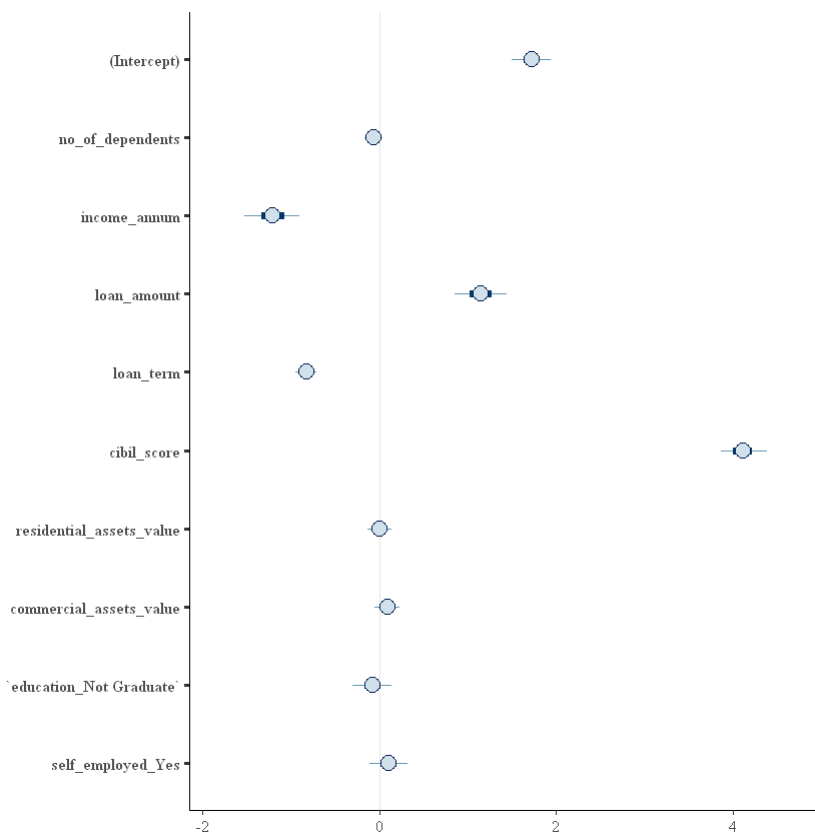
```
par(mfrow = c(2, 2))

plot(
  fit_bayes,
  pch = 19,
  col = "blue",
  main = "Bayesian Logistic Regression"
)

par(mfrow = c(1, 1)) # reset layout
```

Warning message:

"The following arguments were unrecognized and ignored: pch, col, main"



```
library(bayesplot)

pp_check(fit_bayes, type = "dens_overlay")
```

This is bayesplot version 1.15.0

- Online documentation and vignettes at mc-stan.org/bayesplot
- bayesplot theme set to `bayesplot::theme_default()`
 - * Does `_not_` affect other `ggplot2` plots
 - * See `?bayesplot_theme_set` for details on theme setting

Warning message:

"The following arguments were unrecognized and ignored: type"

